

FastAPI + Pydantic

রিকোয়েস্ট হ্যান্ডলিং ও HTTP মেথড

GET, POST, Path Parameters, Query Parameters এবং Response Model — সম্পূর্ণ বাংলায়
বিস্তারিত নোট

FastAPI

Pydantic v2

GET Method

POST Method

Python 3.10+

📋 বিষয়সূচি

- | | |
|-------------------------------|--------------------------|
| 01 সহজ ভাষায় পরিচিতি | 02 বিস্তারিত ব্যাখ্যা |
| 03 Theory ও HTTP স্পেসিফিকেশন | 04 সম্পূর্ণ Code Example |
| 05 Visual Diagram | 06 Real-world Use Cases |
| 07 Pros & Cons | 08 Common Mistakes |
| 09 Related Topics | 10 Memory Tricks |

01 🎯 সহজ ভাষায় পরিচিতি (Intuition)

📅 বাস্তব জীবনের উদাহরণ — দোকানের উদাহরণ

ধরো তুমি একটি মোবাইল স্টোরের মালিক। তোমার দোকানে কেউ আসলে তুমি কয়েক রকম কাজ করো:

📖 **GET:** কেউ জিজ্ঞেস করলো "আপনার কাছে কোন কোন মোবাইল আছে?" — তুমি শুধু তথ্য দিলে, কিছু পরিবর্তন হলো না

✍️ **POST:** কেউ নতুন অর্ডার দিলো — তুমি সেটা তোমার খাতায় লিখে রাখলে, নতুন কিছু তৈরি হলো

🔍 **Path Param:** কেউ বললো "অর্ডার নম্বর ১২৩ কোথায়?" — নম্বর দিয়ে নির্দিষ্ট জিনিস খোঁজা

🔍 **Query Param:** কেউ বললো "১০,০০০ টাকার নিচে Samsung মোবাইল আছে?" — ফিল্টার করা

FastAPI হলো একটি আধুনিক Python web framework যা দিয়ে তুমি দ্রুত API বানাতে পারো। আর **Pydantic** হলো data validation এর টুল — সে নিশ্চিত করে যে আসা ডাটা সঠিক ফরম্যাটে আছে কিনা।

💡 সমস্যা যা সমাধান হয়: আগে Python API বানাতে গেলে manually data validation, docs generation সব করতে হতো। FastAPI + Pydantic মিলে এগুলো automatic করে দেয়, ফলে কোড কম লিখতে হয় এবং bugs কম হয়।

02 বিস্তারিত ব্যাখ্যা (Deep Explanation)

◆ চারটি Core Method বিস্তারিত

GET

ডাটা পড়া (**Read Only**)

Server থেকে তথ্য আনার জন্য ব্যবহার হয়। এই method কোনো data পরিবর্তন করে না। যেমন: সব ইউজার দেখা, একটি নির্দিষ্ট প্রোডাক্টের তথ্য দেখা। URL-এ directly data পাঠানো যায়। **Idempotent** — একই request বারবার পাঠালে একই ফলাফল পাবে।

POST

নতুন ডাটা তৈরি (**Create**)

নতুন resource তৈরি করার জন্য। Request Body তে JSON data পাঠানো হয়। Pydantic model দিয়ে সেই body validate করা হয়। যেমন: নতুন user create, নতুন blog post লেখা। **Not Idempotent** — একই request বারবার পাঠালে বারবার নতুন resource তৈরি হবে।

PATH PARAM

URL এর ভেতরে **Variable**

URL path এর একটি অংশ যা dynamic। যেমন `/users/{user_id}` — এখানে `{user_id}` হলো Path Parameter। FastAPI স্বয়ংক্রিয়ভাবে এটি extract করে function parameter হিসেবে দেয়।

QUERY

URL এর পরে **? দিয়ে**

URL এর শেষে `?` দিয়ে optional filters পাঠানো হয়। যেমন `/users?skip=0&limit=10`। এগুলো optional হতে পারে এবং default value থাকতে পারে। সার্চ, ফিল্টার, pagination এ ব্যবহার হয়।

◆ Pydantic Model — ডাটার গার্ডিয়ান

Pydantic কীভাবে কাজ করে?

1

Request আসে

— Client JSON পাঠায়

2

Pydantic ধরে

— সে JSON টাকে Python class এ convert করে

3

Validation হয়

— প্রতিটি field এর type, length, pattern check করে

4

Error হলে

— স্বয়ংক্রিয় 422 response পাঠায়

5

সঠিক হলে

— তোমার function এ পরিষ্কার Python object হিসেবে পৌঁছায়

Response Model — তথ্য লুকানোর কৌশল

Database এ user এর password থাকে, কিন্তু client কে কখনো password দেওয়া উচিত না। Response Model দিয়ে তুমি নির্ধারণ করো কোন কোন field response এ যাবে। যে field নেই সে গুলো FastAPI লুকিয়ে ফেলে।

03

**Theory ও HTTP** স্পেসিফিকেশন

HTTP METHOD	CRUD OPERATION	STATUS CODE	IDEMPOTENT?	BODY?
GET	Read	200 OK	✓ হ্যাঁ	✗ না
POST	Create	201 Created	✗ না	✓ হ্যাঁ
PUT	Update (Full)	200 OK	✓ হ্যাঁ	✓ হ্যাঁ
PATCH	Update (Partial)	200 OK	✓ হ্যাঁ	✓ হ্যাঁ
DELETE	Delete	204 No Content	✓ হ্যাঁ	✗ না

URL Structure সূত্র

```
https://api.example.com/users/{user_id}?skip=0&limit=10
```



Pydantic Validation Rules

VALIDATOR	ব্যবহার	উদাহরণ
min_length	ন্যূনতম দৈর্ঘ্য	str, min_length=3
max_length	সর্বোচ্চ দৈর্ঘ্য	str, max_length=50
ge / le	Greater/Less than Equal	int, ge=0, le=120
gt / lt	Greater/Less than	float, gt=0
pattern	Regex pattern	str, pattern=r'^\d{11}\$'
Optional[T]	Optional field	Optional[str] = None

04

সম্পূর্ণ Code Example (Python)

✓ নিচের সমস্ত কোড নিজে রান করে ভেরিফাই করা হয়েছে। `TestClient` দিয়ে সব endpoint test করা হয়েছে।

PYTHON

Pydantic Models তৈরি

```
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel, field_validator
from typing import Optional, List

app = FastAPI()

# =====
# PYDANTIC MODELS - ডাটার রূপান্তর
```

```
# =====

class UserCreate(BaseModel):
    # এই model দিয়ে POST request এর body validate হবে
    name: str          # অবশ্যই string হতে হবে
    email: str          # অবশ্যই string হতে হবে
    age: int            # অবশ্যই integer হতে হবে
    bio: Optional[str] = None # optional — না দিলেও চলবে
```

```
@field_validator('age')
@classmethod
def validate_age(cls, v):
    # Custom validation: বয়স ০ থেকে ১২০ এর মধ্যে হতে হবে
    if v < 0 or v > 120:
        raise ValueError('বয়স ০-১২০ এর মধ্যে হতে হবে')
    return v
```

```
class UserResponse(BaseModel):
    # Response Model — শুধু এই field গুলো client দেখবে
    # password এবং age এখানে নেই → স্বয়ংক্রিয়ভাবে লুকানো হবে
    id: int
    name: str
    email: str
```

```
class UserInDB(BaseModel):
    # Database এ যা থাকে (age, password সহ)
    id: int
    name: str
    email: str
    age: int
    password: str = "hashed_secret" # sensitive তথ্য
```

```
# In-memory database (বাস্তবে MongoDB/PostgreSQL ব্যবহার হবে)
fake_db: dict = {}
next_id = 1
```

PYTHON

GET Method — ডাটা পড়া

```
# =====
# GET /users — সব ইউজার দেখো
# Query Parameters: skip (কতটা বাদ দেবো) & limit (কতটা দেখাবো)
# =====

@app.get("/users", response_model=List[UserResponse])
def get_all_users(
    skip: int = 0,          # default: 0 থেকে শুরু
    limit: int = Query(default=10, le=100) # default 10, max 100
```

```

):
    # fake_db থেকে list বানাই, তারপর slice করি
    users = list(fake_db.values())
    return users[skip : skip + limit]

# =====
# GET /users/{user_id} - নির্দিষ্ট ইউজার
# Path Parameter: user_id URL থেকে আসে
# =====

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int): # FastAPI স্বয়ংক্রিয়ভাবে URL থেকে extract করে
    if user_id not in fake_db:
        # 404 Not Found error পাঠাও
        raise HTTPException(status_code=404, detail="ইউজার পাওয়া যায়নি")
    return fake_db[user_id]

# =====
# GET /search - নাম দিয়ে সার্চ
# Query Parameter: q = search keyword
# =====

@app.get("/search")
def search_users(
    q: str = Query(min_length=2, description="সার্চ কি-ওয়ার্ড")
):
    # নাম এ query আছে কিনা চেক করো
    results = [u for u in fake_db.values() if q.lower() in u['name'].lower()]
    return {"query": q, "count": len(results), "results": results}

```

PYTHON

POST Method — নতুন ডাটা তৈরি

```

# =====
# POST /users - নতুন ইউজার তৈরি করো
# request body: UserCreate model (Pydantic validate করবে)
# response: UserResponse model (password লুকানো থাকবে)
# =====

@app.post("/users", response_model=UserResponse, status_code=201)
def create_user(user: UserCreate):
    # user এখন পরিস্কার Python object, validation হয়ে গেছে
    global next_id

    # Database model তৈরি করো (password সহ)
    new_user = UserInDB(
        id=next_id,
        **user.model_dump() # Pydantic object → dict → unpack
    )

```

```
# "Database" তে save করো
fake_db[next_id] = new_user.model_dump()
next_id += 1

# UserResponse model দিয়ে return করো
# FastAPI স্বয়ংক্রিয়ভাবে password ও age বাদ দিয়ে দেবে
return fake_db[next_id - 1]
```

EXPECTED OUTPUT – TESTCLIENT দিয়ে TEST

```
POST /users (201): {'id': 1, 'name': 'রাফি', 'email': 'rafi@test.com'}
↑ password নেই! response_model কাজ করছে ✓

GET /users (200): [{'id': 1, 'name': 'রাফি', ...}, {'id': 2, ...}]
GET /users/1 (200): {'id': 1, 'name': 'রাফি', 'email': 'rafi@test.com'}
GET /users/999 (404): {'detail': 'ইউজার পাওয়া যায়নি'}
GET /search?q=রাফি: {'query': 'রাফি', 'count': 1, 'results': [...]}
POST invalid age (422): Unprocessable Entity – Pydantic validation error
```

✓ Password লুকানো আছে — Response Model কাজ করছে!
 ✓ সব টেস্ট পাস হয়েছে!

05



Visual Diagram

Request Life Cycle — একটি রিকোয়েস্ট কিভাবে প্রসেস হয়:

CLIENT (Browser/App) | | GET /users/42?include=bio ▼

| FASTAPI ROUTER | | | ① URL Matching: "/users/{user_id}" → user_id = 42 | | ② Query
 Parsing: include = "bio" | | ③ Type Casting: "42" (str) → 42 (int) |

| ▼

| PYDANTIC VALIDATION | | | ✓ user_id: int = 42 → OK | | ✓ include: str = "bio" → OK |
 | ✗ age: int = "hello" → 422 Unprocessable Entity |

| (validation passed) ▼

| YOUR FUNCTION | | | def get_user(user_id: 42, include: "bio"): | | user = db.find(42) #
 ডাটাবেস থেকে আনো | | return user |

| RESPONSE MODEL FILTER | | | UserInDB {id, name, email, age, password} | | ↓
response_model=UserResponse | | Response {id, name, email} ← password ও age লুকানো! |

| ▼ CLIENT পায়: {"id": 42, "name": "মারুফ", "email": "maruf@test.com"}

Path Parameter vs Query Parameter পার্থক্য:

Path Parameter: GET /users/42/posts/7 ↑ ↑ user_id post_id → নির্দিষ্ট resource এর জন্য (required)

Query Parameter: GET /users?skip=0&limit=10&sort=name ↑ ↑ ↑ skip limit sort → ফিল্টার ও pagination এর জন্য (optional)

06



Real-world Use Cases

১. 🛒 E-commerce API (Amazon/Daraz)

GET /products?category=phone&max_price=20000&page=2 — Query params দিয়ে ফিল্টার করা। GET /products/{product_id} — নির্দিষ্ট প্রোডাক্ট দেখা। POST /orders — Pydantic দিয়ে order data validate করে নতুন order তৈরি।

২. 🏥 Medical API

GET /patients/{patient_id}/reports — Path param দিয়ে নির্দিষ্ট রোগীর রিপোর্ট। Pydantic দিয়ে blood pressure, age এর validation — অসম্ভব value (যেমন age: -5) automatic reject। Response Model দিয়ে doctor এর personal info লুকানো।

৩. 🔍 Smart Cooking System (তোমার Project!)

POST /cook — Pydantic model দিয়ে dish name, portion size, spice level validate করা। GET /recipes/{recipe_id} — নির্দিষ্ট recipe এর ingredient list। GET /status?session_id=xyz — রান্নার বর্তমান status।

৪. Social Media API (Instagram/Twitter)

GET /users/{username}/posts?limit=20 — Path + Query মিলিয়ে ব্যবহার। **POST /posts** — নতুন পোস্ট তৈরি, Pydantic দিয়ে content length check। Response Model দিয়ে private account এর sensitive data লুকানো।

৫. Banking API

POST /transfer — Pydantic দিয়ে amount (gt=0), account number (pattern) validation। **GET /transactions?from=2024-01&to=2024-12** — Date range query param। Response Model দিয়ে account number masking।

07



Pros & Cons

✓ সুবিধা

- ✓ Automatic data validation — আলাদা করে if-else লিখতে হয় না
- ✓ Auto-generated API docs (/docs) — Swagger UI বিনামূল্যে
- ✓ Type hints দিয়ে editor auto-completion
- ✓ Response Model দিয়ে sensitive data সুরক্ষা সহজ
- ✓ Async support — high performance
- ✓ 422 error automatic — সুন্দর error message
- ✓ Python ecosystem — সব library ব্যবহার করা যায়

✗ অসুবিধা

- ✗ Pydantic v1 থেকে v2 তে বড় পরিবর্তন — migration pain
- ✗ Complex nested validation কঠিন হতে পারে
- ✗ Cold start time Django/Flask এর চেয়ে বেশি
- ✗ Production deployment এ ASGI server (uvicorn) দরকার
- ✗ নতুনদের জন্য async concept বুঝতে সময় লাগে
- ✗ Built-in admin interface নেই (Django এর মতো)



❌ ভুল ১: **Response Model** ছাড়া **sensitive data** পাঠানো
password, token সরাসরি return করা — `response_model=UserResponse` ব্যবহার করো যেখানে sensitive field নেই।

❌ ভুল ২: **Path Parameter** আগে, **Query Parameter** পরে না লেখা
`/users?id=42/{name}` লেখা — সঠিক হলো `/users/{id}?name=xyz` | Path param সবসময় URL path এ থাকবে।

❌ ভুল ৩: **Pydantic v1** এর **syntax** **Pydantic v2** তে ব্যবহার করা
v1: `@validator`, `.dict()` — v2 তে: `@field_validator`, `.model_dump()` | FastAPI 0.100+ ব্যবহার করলে Pydantic v2 ধরে নাও।

⚠️ সতর্কতা: **POST request** এ **GET** এর মতো **URL** এ **data** পাঠানো
Password, email কখনো URL এ পাঠিও না। এগুলো request body তে পাঠাও — নইলে browser history ও server log এ দেখা যাবে।

⚠️ সতর্কতা: **Optional field** এ **None check** না করা
`Optional[str]` মানে সেটা None হতে পারে। function এ `if user.bio is not None:` check করতে হবে।

💡 টিপস: একই **route** এ **Path** এবং **Query** একসাথে ব্যবহার করা যায়
`GET /users/{user_id}/posts?skip=0&limit=5` — FastAPI স্বয়ংক্রিয়ভাবে বুঝবে কোনটা Path, কোনটা Query।



আগে জানা দরকার ছিল

- Python Type Hints (`str`, `int`, `List`)
- HTTP Basics (Request/Response cycle)
- JSON ফরম্যাট

পরে শেখা উচিত

- FastAPI Dependency Injection
- Authentication (JWT, OAuth2)
- Database integration (SQLAlchemy)

→ REST API কি?

→ Python class ও OOP basics

→ Async/Await in FastAPI

→ Background Tasks

→ WebSockets with FastAPI

10



Memory Tricks

GET = দেখা, POST = বানানো

GET দিয়ে ডাটা **দেখি** (কিছু বদলায় না) | POST দিয়ে ডাটা **বানাই** (নতুন কিছু তৈরি হয়)

Path = বাড়ির ঠিকানা, Query = খোঁজার শর্ত

`/users/42` → ৪২ নম্বর বাড়িতে যাও (Path)

`?name=রাফি` → রাফি নামের কাউকে খোঁজো (Query)

Response Model = সার্বভৌম ফিল্টার

DB তে যাই থাক — Response Model এর বাইরে কিছু যাবে না। Password সবসময় Response Model এ রেখো না!

 এক লাইনে সারসংক্ষেপ

FastAPI তে **GET** = তথ্য পড়ো, **POST** = তথ্য তৈরি করো; **Path Param** = URL এর ভেতরে নির্দিষ্ট ID, **Query Param** = URL এর পরে ফিল্টার; আর **Pydantic** = ডাটার গার্ডিয়ান যে validation ও security দেয়।

